

The fact that the code doesn't stop immediately when the base case is reached will be important to keep in mind for the factorial calculation in the next chapter. Remember, any code after the recursive case will still have to run.

At this point, you might be thinking that the recursive `countDownAndUp()` function is overengineered and difficult to follow. Why not, instead, use an iterative solution to print numbers? An *iterative* approach, which uses loops to repeat a task until it's done, is usually thought of as the opposite of recursion.

Whenever you find yourself asking, "Wouldn't using a loop be easier?" the answer is almost certainly "Yes," and you should avoid the recursive solution. Recursion can be tricky for both beginner and experienced programmers, and recursive code isn't automatically "better" or "more elegant" than iterative code. Readable, easy-to-understand code is more important than any supposed elegance that recursion provides. However, on some occasions an algorithm cleanly maps to a recursive approach. Algorithms that involve tree-like data structures and require backtracking are especially suited for recursion. These ideas are further explored in [Chapter 2](#).

Summary

Recursion often confuses new programmers, but it is built on the simple idea that a function can call itself. Every time a function call is made, a new frame object with information related to the call (such as local variables and a return address for the execution to move to when the function returns) is added to the call stack. The call stack, being a stack data structure, can be altered only by having data added to or removed from its "top." This is called *pushing to* and *popping from* the stack, respectively.

The call stack is handled by the program implicitly, so there is no call stack variable. Calling a function pushes a frame object to the call stack, and returning from a function pops a frame object from the call stack.

Recursive functions have recursive cases, those in which a recursive call is made, and base cases, those where the function simply returns. If there is no base case or a bug prevents a base case from being run, the execution causes a stack overflow that crashes the program.

Recursion is a useful technique, but recursion doesn't automatically make code "better" or more "elegant." This idea is explored more in the next chapter.

Further Reading

You can find other introductions to recursion in my 2018 North Bay Python conference talk, "Recursion for Beginners: A Beginner's Guide to Recursion" at <https://youtu.be/AfBqVVKg4GE>. The YouTube channel Computerphile also introduces recursion in its video "What on Earth is Recursion?" at <https://youtu.be/Mo9NEXXIVHc>. Finally, V. Anton Spraul talks about recursion in his book *Think Like a Programmer* (2012, No Starch Press) and in his video